

BORAMA HARDWARE

Backend Database Schema

Tables • Relationships • Auth • RLS • Functions • Triggers • API

Complete Supabase/PostgreSQL specification for the Borama Hardware Store platform

Version 1.0 • July 2025 • Confidential

Database	PostgreSQL 15 (Supabase managed)
Extensions	pgvector, pg_cron, uuid-oss, pg_trgm
Auth	Supabase Auth — phone OTP via Twilio
Currency	All monetary values stored in SLSH (integer). USD derived at runtime.
Roles	customer cashier inventory_manager admin
Bilingual	All user-facing text stored as _en and _so column pairs

1. Schema Overview

The database is organized into 8 functional groups. All tables live in the public schema unless noted. Every table uses a UUID primary key generated by `gen_random_uuid()` except where sequence-based human-readable IDs are required (order numbers, POS transaction numbers).

Group	Tables	Purpose
Core / Config	<code>profiles</code> , <code>locations</code> , <code>location_hours</code> , <code>exchange_rates</code> , <code>mobile_money_settings</code> , <code>banners</code>	User identity, branch config, currency rate, payment providers, promotional banners
Product Catalog	<code>categories</code> , <code>products</code> , <code>product_images</code> , <code>product_variants</code> , <code>product_embeddings</code>	Full product hierarchy including variants, images, and AI vector embeddings
Inventory	<code>inventory</code> , <code>stock_movements</code> , <code>inventory_alerts</code> , <code>stocktakes</code> , <code>stocktake_items</code>	Per-location stock levels, movement history, low-stock alerts, periodic count sheets
Commerce	<code>discount_codes</code> , <code>discount_code_uses</code> , <code>orders</code> , <code>order_items</code> , <code>payment_transactions</code>	Online customer orders, discount application, mobile-money payment tracking
Loyalty	<code>loyalty_tiers</code> , <code>loyalty_cards</code> , <code>loyalty_transactions</code>	Points earning/redemption, tier calculation, card lifecycle
POS	<code>pos_sessions</code> , <code>pos_transactions</code> , <code>pos_transaction_items</code> , <code>pos_payment_splits</code> , <code>parked_transactions</code>	In-store point-of-sale, shift sessions, split payments, offline-safe parked carts
Returns	<code>returns</code> , <code>return_items</code>	Customer return requests for both online orders and POS transactions
AI & System	<code>ai_forecasts</code> , <code>audit_log</code> , <code>sms_logs</code>	Gemini-generated reorder forecasts, full-row audit trail, SMS delivery log

Global Conventions

- **Primary keys:** uuid DEFAULT `gen_random_uuid()` on every table.
- **Timestamps:** `created_at` TIMESTAMPTZ DEFAULT `now()` on all tables. `updated_at` on mutable tables, maintained by trigger.
- **Soft deletes:** `is_active` BOOLEAN DEFAULT `true` used instead of physical DELETE on catalog and config tables.
- **Monetary values:** All prices stored as BIGINT in Somali Shillings (SLSH). USD amounts are never persisted; they are calculated at runtime by dividing by the current exchange rate.
- **Bilingual columns:** Every user-facing string has an `_en` and `_so` variant (e.g. `name_en`, `name_so`).
- **Audit log:** A generic trigger records full JSONB row snapshots (`old_data`, `new_data`) for all admin-modifiable tables.
- **RLS:** Row-Level Security is ENABLED on every table. Service-role key bypasses RLS for server-side operations only.

Currency storage rule: price_slsh, cost_price_slsh, subtotal_slsh, total_slsh, and all other monetary columns are BIGINT representing whole Somali Shillings. The application divides by exchange_rates.usd_to_slsh_rate at render time to show USD. The exchange rate at the moment of order creation is snapshotted into orders.exchange_rate_at_order so historical USD totals remain consistent.

2. Authentication & Authorization

2.1 Authentication Flow — Supabase Phone OTP

Supabase Auth handles all identity. Phone numbers are the sole identifier. No email/password auth is used.

Step	Action	Implementation
1	User enters phone number (Somaliland format, e.g. +252 63 XXXXXXXX)	Client calls <code>supabase.auth.signInWithOtp({ phone })</code>
2	Supabase triggers Twilio — 6-digit OTP SMS sent to phone	Twilio account configured in Supabase Auth settings
3	User enters OTP code	Client calls <code>supabase.auth.verifyOtp({ phone, token, type: 'sms' })</code>
4	On success — <code>auth.users</code> row created (first time) or session returned	Supabase returns <code>access_token</code> + <code>refresh_token</code>
5	Database trigger fires on <code>auth.users</code> INSERT	<code>auto_create_profile()</code> trigger creates <code>profiles</code> row with <code>role = 'customer'</code>
6	Admin assigns elevated role via Admin Portal	<code>UPDATE profiles SET role = 'cashier' 'inventory_manager' 'admin' WHERE user_id = ?</code>
7	All subsequent requests include Authorization: Bearer { <code>access_token</code> }	Supabase RLS uses <code>auth.uid()</code> and claims to enforce per-row permissions

2.2 Role System

Role is stored in `profiles.role`. The application reads this value on every authenticated request and passes it to RLS policies via a custom JWT claim set by a Supabase Auth hook.

Role Value	Display Name	Portals Accessed	Key Permissions
customer	Customer	Customer Store (web/PWA)	Browse, order, view own orders/loyalty, use chatbot
cashier	Cashier / Staff	POS Terminal, Staff Portal	Run POS sessions, confirm mobile money payments, process pickups, initiate returns
inventory_manager	Inventory Manager	Inventory Portal, Staff Portal	All cashier permissions + manage stock, run stocktakes, view forecasts, receive stock
admin	Admin / Owner	All portals including Admin	Full access: manage users, products, pricing, discounts, locations, analytics, roles

2.3 Custom JWT Claim Hook

A Supabase Auth hook enriches the JWT with the user's role so RLS policies can use it without querying the profiles table on every row read:

```
-- Supabase Auth Hook: customize_access_token
CREATE OR REPLACE FUNCTION auth.customize_access_token(event JSONB)
RETURNS JSONB LANGUAGE plpgsql AS $$
DECLARE
  _role TEXT;
BEGIN
  SELECT role INTO _role FROM public.profiles
  WHERE user_id = (event->>'user_id')::uuid;
  RETURN jsonb_set(event, '{claims,user_role}', to_jsonb(_role));
END;
$$;

-- Usage in RLS policies:
-- (auth.jwt() ->> 'user_role') = 'admin'
```

2.4 RLS Helper Functions

These utility functions are referenced by RLS policies across all tables:

```
-- Returns current user's role from JWT claim
CREATE OR REPLACE FUNCTION auth.user_role()
RETURNS TEXT LANGUAGE sql STABLE AS $$
SELECT (auth.jwt() ->> 'user_role')::text;
$$;

-- Returns current user's location_id from profiles
CREATE OR REPLACE FUNCTION auth.user_location_id()
RETURNS UUID LANGUAGE sql STABLE SECURITY DEFINER AS $$
SELECT location_id FROM public.profiles WHERE user_id = auth.uid();
$$;

-- Convenience booleans
CREATE OR REPLACE FUNCTION auth.is_admin()
RETURNS BOOLEAN LANGUAGE sql STABLE AS $$
SELECT auth.user_role() = 'admin';
$$;

CREATE OR REPLACE FUNCTION auth.is_staff()
RETURNS BOOLEAN LANGUAGE sql STABLE AS $$
SELECT auth.user_role() IN ('cashier','inventory_manager','admin');
$$;
```

3. Core & Configuration Tables

profiles

Extends auth.users. One row per authenticated user.

Column	Type	Constraints / Default	Description
user_id	UUID	PK, FK auth.users(id) ON DELETE CASCADE	Matches Supabase auth identity
phone	TEXT	NOT NULL, UNIQUE	E.164 phone number, e.g. +252631234567
role	TEXT	NOT NULL DEFAULT 'customer', CHECK role IN (...)	customer cashier inventory_manager admin
full_name	TEXT	NOT NULL	Display name, required on first sign-in
location_id	UUID	FK locations(id), NULLABLE	Assigned branch for staff; NULL for customers
preferred_language	TEXT	NOT NULL DEFAULT 'en', CHECK IN ('en','so')	UI language preference
preferred_currency	TEXT	NOT NULL DEFAULT 'SLSH', CHECK IN ('SLSH','USD')	Display currency preference
is_active	BOOLEAN	NOT NULL DEFAULT true	Set false to suspend account without deleting
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Updated by trigger on any change

RLS: Users read and update their own row. Staff read any profile at their location. Admin reads and updates all.

locations

Physical store branches in Borama.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
name_en	TEXT	NOT NULL	English branch name, e.g. 'Main Branch'
name_so	TEXT	NOT NULL	Somali branch name
address	TEXT	NOT NULL	Street address in Borama
phone	TEXT		Branch contact phone
is_active	BOOLEAN	NOT NULL DEFAULT true	Only active locations appear in customer UI
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Public SELECT (any authenticated or anonymous user can read active locations). Only admin INSERT/UPDATE/DELETE.

location_hours

Operating hours per branch per day. Supports prayer break windows.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
location_id	UUID	NOT NULL, FK locations(id) ON DELETE CASCADE	
day_of_week	SMALLINT	NOT NULL, CHECK (0..6)	0=Sunday through 6=Saturday
open_time	TIME	NOT NULL	e.g. 08:00
close_time	TIME	NOT NULL	e.g. 21:00
is_closed	BOOLEAN	NOT NULL DEFAULT false	True on Friday afternoon or holidays
has_prayer_break	BOOLEAN	NOT NULL DEFAULT false	Enables prayer break display in UI
prayer_start	TIME	NULLABLE	Prayer break start time
prayer_end	TIME	NULLABLE	Prayer break end time

RLS: Public SELECT. Admin INSERT/UPDATE/DELETE.

exchange_rates

History of USD/SLSH exchange rates. Most recent row is current rate.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
usd_to_slsh_rate	NUMERIC(10,2)	NOT NULL	e.g. 570.00 means 1 USD = 570 SLSH
set_by	UUID	FK profiles(user_id)	Admin who set this rate
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Acts as valid_from timestamp

RLS: Public SELECT (needed for currency display). Admin INSERT only. No UPDATE or DELETE — new row supersedes previous.

mobile_money_settings

Per-branch mobile money provider configuration.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
location_id	UUID	NOT NULL, FK locations(id) ON DELETE CASCADE	
provider	TEXT	NOT NULL, CHECK IN ('za ad','edahab','evc_plus','sah al')	Mobile money network
merchant_number	TEXT	NOT NULL	Number customers send money to
instructions_en	TEXT	NOT NULL	Step-by-step payment instructions in English
instructions_so	TEXT	NOT NULL	Step-by-step instructions in Somali
is_active	BOOLEAN	NOT NULL DEFAULT true	Toggle off if provider unavailable

RLS: Authenticated SELECT. Admin INSERT/UPDATE/DELETE.

banners

Promotional banners shown on the customer home page. Scope controls which branches see the banner.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
title_en	TEXT	NOT NULL	Banner headline in English
title_so	TEXT	NOT NULL	Banner headline in Somali
image_url	TEXT	NOT NULL	Supabase Storage public URL
cta_text_en	TEXT		Call-to-action button label (English)
cta_text_so	TEXT		Call-to-action button label (Somali)
cta_url	TEXT		Relative path the CTA navigates to
scope_type	TEXT	NOT NULL DEFAULT 'all', CHECK IN ('all','location')	Whether banner applies to all branches or one
scope_location_id	UUID	FK locations(id), NULLABLE	Set when scope_type = 'location'
active_from	TIMESTAMPTZ	NOT NULL	Banner display start
active_until	TIMESTAMPTZ	NOT NULL	Banner display end
sort_order	INTEGER	NOT NULL DEFAULT 0	Lower number = shown first
is_active	BOOLEAN	NOT NULL DEFAULT true	Master on/off toggle
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Public SELECT (filter active_from/active_until in application). Admin INSERT/UPDATE/DELETE.

4. Product Catalog Tables

categories

Self-referencing hierarchy. Top-level rows have parent_id = NULL.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
parent_id	UUID	FK categories(id) ON DELETE SET NULL, NULLABLE	NULL = top-level category
name_en	TEXT	NOT NULL	English category name
name_so	TEXT	NOT NULL	Somali category name
icon_url	TEXT		Supabase Storage URL for category icon
sort_order	INTEGER	NOT NULL DEFAULT 0	Display order within parent
is_active	BOOLEAN	NOT NULL DEFAULT true	
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Public SELECT. Admin INSERT/UPDATE/DELETE.

products

Master product record. Pricing lives on product_variants. One product can have many variants.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
name_en	TEXT	NOT NULL	Product name in English
name_so	TEXT	NOT NULL	Product name in Somali
description_en	TEXT		Full description in English (AI-generated or manual)
description_so	TEXT		Full description in Somali
category_id	UUID	NOT NULL, FK categories(id)	Must be a leaf-level category
brand	TEXT		Brand or manufacturer name
unit	TEXT	NOT NULL DEFAULT 'each'	Unit of sale: each, kg, m, L, box, bag, etc.
sku_base	TEXT	NOT NULL, UNIQUE	Base SKU — variants append suffix (e.g. BOLT-M4-25MM)
cost_price_slsh	BIGINT	NOT NULL DEFAULT 0	Default cost price in SL\$H (overridden per variant)
tags	TEXT[]	NOT NULL DEFAULT '{}'	Searchable tags, e.g. {cement, binding, bag}
is_featured	BOOLEAN	NOT NULL DEFAULT false	Shows on home page featured section

Column	Type	Constraints / Default	Description
is_active	BOOLEAN	NOT NULL DEFAULT true	Hidden from customers when false
created_by	UUID	FK profiles(user_id)	Staff member who created the listing
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Updated by trigger

RLS: Public SELECT on is_active=true rows. Admin/inventory_manager INSERT/UPDATE. Only admin DELETE (soft: set is_active=false).

product_images

Ordered image gallery per product. sort_order=0 is the primary display image.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
product_id	UUID	NOT NULL, FK products(id) ON DELETE CASCADE	
image_url	TEXT	NOT NULL	Supabase Storage full-resolution URL
thumbnail_url	TEXT	NOT NULL	Supabase Storage thumbnail URL (Sharp-processed)
alt_text_en	TEXT		Screen-reader alt text in English
alt_text_so	TEXT		Screen-reader alt text in Somali
sort_order	INTEGER	NOT NULL DEFAULT 0	0 = primary image shown in listings

RLS: Public SELECT. Admin/inventory_manager INSERT/UPDATE/DELETE.

product_variants

A product variant is the purchasable unit. Each variant has its own SKU, price, and stock record. Products with no physical variation still have one default variant.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
product_id	UUID	NOT NULL, FK products(id) ON DELETE CASCADE	
sku	TEXT	NOT NULL, UNIQUE	Full variant SKU, e.g. BOLT-M4-25MM-ZN
price_slsh	BIGINT	NOT NULL	Absolute selling price in SLISH. NOT a delta from base.
cost_price_slsh	BIGINT	NOT NULL	Variant-level cost price in SLISH

Column	Type	Constraints / Default	Description
attributes	JSONB	NOT NULL DEFAULT '{}'	Variant dimensions, e.g. {"Size":"M4","Length":"25mm","Finish":"Zinc"}
image_url	TEXT		Variant-specific image URL (overrides product gallery)
is_active	BOOLEAN	NOT NULL DEFAULT true	Hidden from storefront when false

RLS: Public SELECT on is_active=true. Admin/inventory_manager INSERT/UPDATE/DELETE.

product_embeddings

Stores vector embeddings for semantic search. Separate from products to keep the products table lean. Generated by Gemini text-embedding-004 (768 dimensions).

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
product_id	UUID	NOT NULL, FK products(id) ON DELETE CASCADE	
language	TEXT	NOT NULL, CHECK IN ('en','so','combined')	Which language text was embedded
embedding	vector(768)	NOT NULL	pgvector column — requires CREATE EXTENSION vector
embedding_text	TEXT	NOT NULL	The text that was embedded (for audit/re-embedding)
last_embedded_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
model_version	TEXT	NOT NULL DEFAULT 'text-embedding-004'	Embedding model identifier

Index: CREATE INDEX ON product_embeddings USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

RLS: Authenticated SELECT. Server-side INSERT/UPDATE via service role only.

5. Inventory Tables

inventory

Per-location stock level for each product variant. `quantity_on_hand` is ground truth after confirmed sales. `quantity_reserved` tracks hold placed at checkout, released after 30 minutes if payment not confirmed.

Column	Type	Constraints / Default	Description
<code>id</code>	UUID	PK DEFAULT gen_random_uuid()	
<code>product_id</code>	UUID	NOT NULL, FK products(id) ON DELETE CASCADE	
<code>variant_id</code>	UUID	FK product_variants(id) ON DELETE CASCADE, NULLABLE	NULL = product-level stock (no variants)
<code>location_id</code>	UUID	NOT NULL, FK locations(id) ON DELETE CASCADE	
<code>quantity_on_hand</code>	INTEGER	NOT NULL DEFAULT 0, CHECK >= 0	Confirmed physical stock available for sale
<code>quantity_reserved</code>	INTEGER	NOT NULL DEFAULT 0, CHECK >= 0	Units held at checkout, not yet payment-confirmed
<code>threshold</code>	INTEGER	NOT NULL DEFAULT 5	Low-stock alert fires when on_hand <= threshold
<code>aisle_shelf</code>	TEXT		Physical location in store, e.g. 'A3-Shelf2'
<code>last_restocked_at</code>	TIMESTAMPTZ		Timestamp of most recent positive stock movement
<code>created_at</code>	TIMESTAMPTZ	NOT NULL DEFAULT now()	
<code>updated_at</code>	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Available stock formula: `available_qty = quantity_on_hand - quantity_reserved`. This is what customers see. Always CHECK (`quantity_on_hand - quantity_reserved >= 0`) before allowing checkout. UNIQUE constraint on (`product_id`, `variant_id`, `location_id`).

RLS: Public SELECT on available quantity (`quantity_on_hand - quantity_reserved`). Inventory_manager/admin INSERT/UPDATE for their location. Admin updates any location.

stock_movements

Immutable ledger of every stock change. Never UPDATE or DELETE rows — append only.

Column	Type	Constraints / Default	Description
<code>id</code>	UUID	PK DEFAULT gen_random_uuid()	

Column	Type	Constraints / Default	Description
product_id	UUID	NOT NULL, FK products(id)	
variant_id	UUID	FK product_variants(id), NULLABLE	
location_id	UUID	NOT NULL, FK locations(id)	
movement_type	TEXT	NOT NULL, CHECK IN ('receive','sale','return', 'transfer_in','transfer_out', 'adjustment','stocktake_correction','void')	Type of movement
quantity_change	INTEGER	NOT NULL	Positive = stock in, negative = stock out
reference_id	UUID		FK to source record (order_id, stocktake_id, etc.)
reference_type	TEXT		'order' 'pos_transaction' 'stocktake' 'manual'
notes	TEXT		Optional free-text explanation
performed_by	UUID	FK profiles(user_id)	Staff member who triggered the movement
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Staff SELECT for their location. Admin SELECT all. No UPDATE or DELETE for any role.

inventory_alerts

Generated automatically by trigger when quantity_on_hand drops to or below threshold.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
product_id	UUID	NOT NULL, FK products(id)	
variant_id	UUID	FK product_variants(id), NULLABLE	
location_id	UUID	NOT NULL, FK locations(id)	
alert_type	TEXT	NOT NULL, CHECK IN ('low_stock','out_of_stock')	
is_resolved	BOOLEAN	NOT NULL DEFAULT false	Set true when stock replenished above threshold
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
resolved_at	TIMESTAMPTZ		

RLS: Inventory_manager/admin SELECT for their location. System (trigger) INSERT. Staff UPDATE to resolve.

stocktakes

A stocktake is a full or partial manual count of physical stock at a branch. Must be approved by admin before inventory adjustments are applied.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
location_id	UUID	NOT NULL, FK locations(id)	
status	TEXT	NOT NULL DEFAULT 'draft', CHECK IN ('draft','s ubmitted','approved','reject ed')	
initiated_by	UUID	NOT NULL, FK profiles(user_id)	
submitted_at	TIMESTAMPTZ		
approved_by	UUID	FK profiles(user_id), NULLABLE	
approved_at	TIMESTAMPTZ		
rejection_reason	TEXT		
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Inventory_manager/admin SELECT/INSERT/UPDATE for their location. Only admin can set status='approved'.

stocktake_items

Individual line items within a stocktake — one row per product/variant counted.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
stocktake_id	UUID	NOT NULL, FK stocktakes(id) ON DELETE CASCADE	
product_id	UUID	NOT NULL, FK products(id)	
variant_id	UUID	FK product_variants(id), NULLABLE	
system_quantity	INTEGER	NOT NULL	Quantity on hand at time stocktake was initiated
counted_quantity	INTEGER	NOT NULL	Physical count entered by staff
discrepancy	INTEGER	GENERATED ALWAYS AS (counted_quantity - system_quantity) STORED	Positive = surplus, negative = shortage
notes	TEXT		Optional comment for this line

RLS: Same as parent stocktakes row.

6. Commerce Tables

discount_codes

Admin-created promotional codes. One code per order — no stacking. Loyalty point redemption is tracked separately and is additive with one discount code.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
code	TEXT	NOT NULL, UNIQUE	Case-insensitive code, e.g. SUMMER10
discount_type	TEXT	NOT NULL, CHECK IN (‘percentage’, ‘fixed_slsh’)	
value	NUMERIC(10, 2)	NOT NULL	% value (e.g. 10.00) or fixed SLSH amount
minimum_order_slsh	BIGINT	NOT NULL DEFAULT 0	Order total must reach this before code applies
max_total_uses	INTEGER		NULL = unlimited total redemptions
uses_count	INTEGER	NOT NULL DEFAULT 0	Incremented on each redemption
max_uses_per_customer	INTEGER	NOT NULL DEFAULT 1	How many times one customer can use this code
scope_type	TEXT	NOT NULL DEFAULT ‘all’, CHECK IN (‘all’, ‘category’, ‘product’)	Code applies to entire order or specific scope
scope_id	UUID		category_id or product_id when scope_type != ‘all’
valid_from	TIMESTAMPTZ	NOT NULL	
valid_until	TIMESTAMPTZ	NOT NULL	
is_active	BOOLEAN	NOT NULL DEFAULT true	Master kill switch
created_by	UUID	FK profiles(user_id)	
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Authenticated SELECT only for validation. Admin INSERT/UPDATE/DELETE.

discount_code_uses

Records each application of a discount code to prevent exceeding per-customer limits.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
discount_code_id	UUID	NOT NULL, FK discount_codes(id) ON DELETE CASCADE	
customer_id	UUID	NOT NULL, FK profiles(user_id)	
order_id	UUID	NOT NULL, FK orders(id)	
used_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Unique constraint: (discount_code_id, customer_id, order_id) prevents duplicate records.

RLS: Server-side INSERT only (via API route). No direct client access.

orders

Online orders placed by customers. POS transactions are tracked separately in pos_transactions. Stock is reserved on order submit and released or decremented on payment confirmation.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
order_number	TEXT	NOT NULL, UNIQUE	Human-readable: BH-2025-00001, generated by function
customer_id	UUID	NOT NULL, FK profiles(user_id)	
location_id	UUID	NOT NULL, FK locations(id)	Branch for pickup
status	TEXT	NOT NULL DEFAULT 'pending_payment', CHECK IN ('pending_paym ent','payment_submitted', ' payment_confirmed','ready _for_pickup', 'completed','cancelled')	Order lifecycle state
payment_method	TEXT	NOT NULL, CHECK IN ('za ad','edahab','evc_plus','sah al','cash_on_pickup')	
payment_status	TEXT	NOT NULL DEFAULT 'pending', CHECK IN ('pen ding','submitted','confirmed' , 'failed')	
subtotal_slsh	BIGINT	NOT NULL	Sum of item totals before discounts
discount_amount_slsh	BIGINT	NOT NULL DEFAULT 0	Reduction from discount code

Column	Type	Constraints / Default	Description
loyalty_discount_slsh	BIGINT	NOT NULL DEFAULT 0	Reduction from loyalty point redemption
total_slsh	BIGINT	NOT NULL	subtotal - discount_amount - loyalty_discount
discount_code_id	UUID	FK discount_codes(id), NULLABLE	NULL if no code applied
loyalty_points_redeemed	INTEGER	NOT NULL DEFAULT 0	Points used in this order
loyalty_points_earned	INTEGER	NOT NULL DEFAULT 0	Points awarded after order completion
pickup_code	TEXT	UNIQUE, NULLABLE	6-char alphanumeric shown to customer for in-store pickup
payment_reference	TEXT		Transaction ID customer provides from mobile money
cancellation_reason	TEXT		
notes	TEXT		Customer delivery/special instructions
exchange_rate_at_order	NUMERIC(10,2)	NOT NULL	USD rate snapshot at order creation time
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Status transitions: pending_payment → payment_submitted (customer submits ref) → payment_confirmed (staff confirms) → ready_for_pickup (stock confirmed) → completed (customer picks up). Any state → cancelled (customer or staff).

RLS: Customers SELECT/INSERT own orders. Staff SELECT orders at their location. Admin SELECT all. Staff UPDATE status. Only server-side UPDATE for payment fields.

order_items

Line items for each order. Product name and SKU are denormalized at time of order to preserve historical accuracy if the product is later renamed or removed.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
order_id	UUID	NOT NULL, FK orders(id) ON DELETE CASCADE	
product_id	UUID	FK products(id), SET NULL on delete	
variant_id	UUID	FK product_variants(id), SET NULL on delete	
product_name_en	TEXT	NOT NULL	Snapshot of name at time of order
product_name_so	TEXT	NOT NULL	Snapshot of Somali name at time of order
sku	TEXT	NOT NULL	Snapshot of SKU at time of order
quantity	INTEGER	NOT NULL, CHECK > 0	
unit_price_slsh	BIGINT	NOT NULL	Price per unit at time of order
total_price_slsh	BIGINT	NOT NULL	quantity * unit_price_slsh

RLS: Inherits from orders — same policies.

payment_transactions

Records payment attempts and confirmations for online orders. Staff manually confirm mobile money payments by matching reference numbers.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
order_id	UUID	FK orders(id) ON DELETE CASCADE, NULLABLE	
pos_transaction_id	UUID	FK pos_transactions(id), NULLABLE	One of order_id or pos_transaction_id must be set
payment_method	TEXT	NOT NULL	zaad edahab evc_plus sahal cash
amount_slsh	BIGINT	NOT NULL	Amount paid in SLISH
status	TEXT	NOT NULL DEFAULT 'pending', CHECK IN ('pen ding','confirmed','failed')	
transaction_refe rence	TEXT		Mobile money transaction ID from customer
confirmed_by	UUID	FK profiles(user_id), NULLABLE	Staff who verified payment
confirmed_at	TIMESTAMPTZ		
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Customers SELECT own records. Staff SELECT for their location. Admin SELECT all. Staff UPDATE status to 'confirmed'. Server-side INSERT only.

7. Loyalty Tables

loyalty_tiers

Admin-configurable tier thresholds and discount rates. Three tiers: bronze, silver, gold. A customer's tier is recalculated any time their lifetime points change.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
tier_name	TEXT	NOT NULL, UNIQUE, CHECK IN ('bronze','silver','gold')	
tier_name_so	TEXT	NOT NULL	Somali display name
min_lifetime_points	INTEGER	NOT NULL	Minimum lifetime points to qualify for this tier
discount_percentage	NUMERIC(4,2)	NOT NULL	% discount applied when redeeming points at checkout
updated_by	UUID	FK profiles(user_id)	Last admin to change tier config
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Seed data: bronze (0+ pts, 2%), silver (500+ pts, 5%), gold (2000+ pts, 10%).

RLS: Public SELECT. Admin UPDATE only (tiers are never created or deleted after seed).

loyalty_cards

One loyalty card per customer, created automatically by trigger on new customer profile. Tracks current spendable points and lifetime accumulated points for tier calculation.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
customer_id	UUID	NOT NULL, UNIQUE, FK profiles(user_id) ON DELETE CASCADE	One card per customer
card_number	TEXT	NOT NULL, UNIQUE	Unique display number, e.g. BH-LC-000001
current_points	INTEGER	NOT NULL DEFAULT 0, CHECK >= 0	Points available to redeem
lifetime_points	INTEGER	NOT NULL DEFAULT 0, CHECK >= 0	Total points ever earned — never decremented
current_tier	TEXT	NOT NULL DEFAULT 'bronze', CHECK IN ('bronze', 'silver', 'gold')	Updated by trigger when lifetime_points changes
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Customers SELECT own card. Staff SELECT any card (needed to look up by phone at POS). Admin SELECT all.

loyalty_transactions

Immutable ledger of all point changes. Append-only — no UPDATE or DELETE.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
loyalty_card_id	UUID	NOT NULL, FK loyalty_cards(id)	
transaction_type	TEXT	NOT NULL, CHECK IN (‘earn’,‘redeem’,‘adjust’)	earn = points added; redeem = points spent; adjust = admin correction
points	INTEGER	NOT NULL	Positive for earn/adjust-up, negative for redeem/adjust-down
reference_type	TEXT		‘order’ ‘pos_transaction’ ‘manual_adjustment’
reference_id	UUID		FK to source record
notes	TEXT		Required for manual adjustments
performed_by	UUID	FK profiles(user_id)	System (trigger) or admin for manual adjustments
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Points earning rule: 1 point per 1,000 SL\$H spent (rounded down). Applied after discounts.

RLS: Customers SELECT own transactions. Staff SELECT any (for POS lookup). Admin SELECT all. No UPDATE or DELETE for any role — system trigger INSERT only.

8. Point-of-Sale (POS) Tables

pos_sessions

A cashier opens a session at shift start with their starting cash float. All transactions during a shift link to the open session. Session is closed at shift end.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
cashier_id	UUID	NOT NULL, FK profiles(user_id)	
location_id	UUID	NOT NULL, FK locations(id)	
status	TEXT	NOT NULL DEFAULT 'open', CHECK IN ('open', 'closed')	
starting_cash_slsh	BIGINT	NOT NULL DEFAULT 0	Cash float at session open
starting_cash_usd	NUMERIC(10,2)	NOT NULL DEFAULT 0	USD float at session open (informational)
ending_cash_slsh	BIGINT		Cash counted at session close
ending_cash_usd	NUMERIC(10,2)		USD cash at session close
cash_variance_slsh	BIGINT	GENERATED ALWAYS AS (ending_cash_slsh - starting_cash_slsh - total_cash_sales_slsh) STORED	Positive = surplus, negative = shortage
total_sales_slsh	BIGINT	NOT NULL DEFAULT 0	Sum of completed transaction totals — updated by trigger
total_cash_sales_slsh	BIGINT	NOT NULL DEFAULT 0	Cash portion of sales — for variance calculation
total_voids_slsh	BIGINT	NOT NULL DEFAULT 0	Sum of voided transaction totals
opened_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
closed_at	TIMESTAMPTZ		

Constraint: A cashier may only have one open session at a time at a given location. UNIQUE constraint on (cashier_id, status) WHERE status = 'open'.

RLS: Cashier SELECT own sessions. Admin SELECT all sessions. Cashier INSERT (open). Cashier UPDATE (close own). Admin UPDATE any.

pos_transactions

Each completed or voided in-store sale. A cashier may optionally link a customer phone to earn loyalty points. Stock is decremented immediately on completed transactions.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
pos_session_id	UUID	NOT NULL, FK pos_sessions(id)	
location_id	UUID	NOT NULL, FK locations(id)	Denormalized for fast location-based queries
transaction_number	TEXT	NOT NULL, UNIQUE	POS-YYYY-NNNNN format, sequential per location
cashier_id	UUID	NOT NULL, FK profiles(user_id)	
customer_id	UUID	FK profiles(user_id), NULLABLE	Linked customer (for loyalty)
customer_phone	TEXT		Phone entered at POS even if customer not registered
status	TEXT	NOT NULL DEFAULT 'completed', CHECK IN ('completed', 'voided')	
subtotal_slsh	BIGINT	NOT NULL	
discount_amount_slsh	BIGINT	NOT NULL DEFAULT 0	
total_slsh	BIGINT	NOT NULL	
loyalty_points_earned	INTEGER	NOT NULL DEFAULT 0	
discount_code_id	UUID	FK discount_codes(id), NULLABLE	
void_reason	TEXT		Required when status = 'voided'
voided_by	UUID	FK profiles(user_id), NULLABLE	Staff who authorized the void
voided_at	TIMESTAMPTZ		
exchange_rate_at_sale	NUMERIC(10,2)	NOT NULL	USD rate at time of transaction
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Cashier INSERT/SELECT for their session. Staff SELECT for their location. Admin SELECT all. Only admin/manager can UPDATE status to 'voided'.

pos_transaction_items

Line items within a POS transaction. Product name and SKU are snapshotted at time of sale.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
pos_transaction_id	UUID	NOT NULL, FK pos_transactions(id) ON DELETE CASCADE	
product_id	UUID	FK products(id), SET NULL on delete	
variant_id	UUID	FK product_variants(id), SET NULL on delete	
product_name_en	TEXT	NOT NULL	Snapshot at time of sale
sku	TEXT	NOT NULL	Snapshot at time of sale
quantity	INTEGER	NOT NULL, CHECK > 0	
unit_price_slsh	BIGINT	NOT NULL	Price at time of sale
total_price_slsh	BIGINT	NOT NULL	quantity * unit_price_slsh

RLS: Same as parent pos_transactions.

pos_payment_splits

Supports split payment — e.g. part cash, part Zaad. One row per payment method used in a transaction.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
pos_transaction_id	UUID	NOT NULL, FK pos_transactions(id) ON DELETE CASCADE	
payment_method	TEXT	NOT NULL, CHECK IN ('ca sh','zaad','edahab','evc_plu s','sahal')	
amount_slsh	BIGINT	NOT NULL	Amount tendered for this method
change_slsh	BIGINT	NOT NULL DEFAULT 0	Change given back (cash only)
transaction_refe rence	TEXT		Mobile money reference (non-cash)
is_confirmed	BOOLEAN	NOT NULL DEFAULT false	True once mobile money verified by staff
confirmed_by	UUID	FK profiles(user_id), NULLABLE	
confirmed_at	TIMESTAMPTZ		

RLS: Same as parent pos_transactions.

parked_transactions

Allows a cashier to save a partially-built cart (e.g. to help another customer) and recall it later. Parked carts auto-expire after 4 hours via pg_cron. No stock is reserved while parked.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
pos_session_id	UUID	NOT NULL, FK pos_sessions(id) ON DELETE CASCADE	
cashier_id	UUID	NOT NULL, FK profiles(user_id)	
location_id	UUID	NOT NULL, FK locations(id)	
cart_data	JSONB	NOT NULL	Full cart state: items, quantities, prices, applied code
parked_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
expires_at	TIMESTAMPTZ	NOT NULL DEFAULT now() + interval '4 hours'	pg_cron deletes expired rows
is_recalled	BOOLEAN	NOT NULL DEFAULT false	Set true when cart is recalled (before deletion)

RLS: Cashier SELECT/INSERT/UPDATE/DELETE own parked transactions. Admin SELECT all.

9. Returns & AI Tables

returns

Customer return requests for both online orders and POS transactions. Refund is issued to original payment method or cash depending on return_items assessment.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
order_id	UUID	FK orders(id), NULLABLE	Set if return is for an online order
pos_transaction_id	UUID	FK pos_transactions(id), NULLABLE	Set if return is for a POS sale
customer_id	UUID	NOT NULL, FK profiles(user_id)	
location_id	UUID	NOT NULL, FK locations(id)	Branch processing the return
status	TEXT	NOT NULL DEFAULT 'pending', CHECK IN ('pending', 'approved', 'rejected')	
refund_method	TEXT	CHECK IN ('original_payment', 'cash', 'store_credit'), NULLABLE	
refund_amount_slsh	BIGINT		Determined by staff on approval
mobile_money_phone	TEXT		Phone to receive refund if mobile money
refund_reference	TEXT		Mobile money transaction ID for refund
rejection_reason	TEXT		
processed_by	UUID	FK profiles(user_id), NULLABLE	Staff who approved or rejected
processed_at	TIMESTAMPTZ		
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Constraint: Exactly one of order_id or pos_transaction_id must be non-null. Enforced by CHECK (order_id IS NOT NULL) != (pos_transaction_id IS NOT NULL).

RLS: Customers INSERT and SELECT own returns. Staff SELECT/UPDATE returns at their location. Admin all.

return_items

Individual items within a return request. Customer uploads photos for damage assessment.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
return_id	UUID	NOT NULL, FK returns(id) ON DELETE CASCADE	
order_item_id	UUID	FK order_items(id), NULLABLE	Set for online order returns
product_id	UUID	FK products(id)	
variant_id	UUID	FK product_variants(id), NULLABLE	
quantity	INTEGER	NOT NULL, CHECK > 0	
reason	TEXT	NOT NULL	Customer's stated reason for return
photo_urls	TEXT[]	NOT NULL DEFAULT '{}'	Supabase Storage URLs of uploaded photos

RLS: Same as parent returns.

ai_forecasts

Inventory reorder forecasts generated by Gemini. Each row is a point-in-time prediction. A pg_cron job triggers weekly regeneration for all products at all locations.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
product_id	UUID	NOT NULL, FK products(id) ON DELETE CASCADE	
variant_id	UUID	FK product_variants(id), NULLABLE	
location_id	UUID	NOT NULL, FK locations(id)	
predicted_stockout_date	DATE	NOT NULL	When stock is predicted to hit zero
recommended_reorder_qty	INTEGER	NOT NULL	Units to order to cover next 30 days
confidence_score	NUMERIC(3, 2)	NOT NULL, CHECK 0..1	Gemini confidence, e.g. 0.85
reasoning_text	TEXT	NOT NULL	Gemini's plain-language explanation
generated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
data_period_days	INTEGER	NOT NULL DEFAULT 90	Days of historical sales data used

RLS: Inventory_manager/admin SELECT for their location. Service role INSERT only.

10. System Tables

audit_log

Immutable audit trail for all admin-modifiable tables. Full JSONB row snapshots are stored for both old and new values. A generic trigger is attached to each audited table.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
table_name	TEXT	NOT NULL	Name of the affected table, e.g. 'products'
record_id	UUID	NOT NULL	Primary key of the affected row
action	TEXT	NOT NULL, CHECK IN ('INSERT','UPDATE','DELETE')	
old_data	JSONB		Full row before change. NULL for INSERT.
new_data	JSONB		Full row after change. NULL for DELETE.
performed_by	UUID	FK profiles(user_id), NULLABLE	NULL for system-triggered changes
performed_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Audited tables: products, product_variants, categories, inventory, discount_codes, locations, profiles (role changes only), loyalty_tiers, mobile_money_settings, banners.

Security: RLS allows admin SELECT only. No UPDATE or DELETE for any role or service role — the table is append-only by policy. A separate Supabase Vault backup exports weekly.

sms_logs

Records every outbound SMS sent via Twilio. Used for debugging delivery failures and preventing duplicate sends.

Column	Type	Constraints / Default	Description
id	UUID	PK DEFAULT gen_random_uuid()	
recipient_phone	TEXT	NOT NULL	E.164 destination phone number
message_text	TEXT	NOT NULL	Full SMS body sent
trigger_event	TEXT	NOT NULL	'order_confirmed' 'payment_received' 'ready_for_pickup' 'otp' etc.
reference_id	UUID		FK to triggering record (order_id, etc.)
reference_type	TEXT		'order' 'return' 'auth'
status	TEXT	NOT NULL DEFAULT 'queued', CHECK IN ('queued', 'sent', 'delivered', 'failed')	
twilio_sid	TEXT		Twilio message SID for status lookup
error_code	TEXT		Twilio error code if failed
attempt_count	INTEGER	NOT NULL DEFAULT 1	Incremented on retry
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

RLS: Admin SELECT only. Server-side INSERT/UPDATE via service role.

11. Database Functions

11.1 generate_order_number()

Generates sequential human-readable order numbers in BH-YYYY-NNNNN format. Called automatically by trigger on orders INSERT.

```
CREATE SEQUENCE IF NOT EXISTS order_number_seq START 1;
CREATE OR REPLACE FUNCTION generate_order_number()
RETURNS TEXT LANGUAGE plpgsql AS $$
BEGIN
RETURN 'BH-' || to_char(now(), 'YYYY') || '-'
|| lpad(nextval('order_number_seq')::text, 5, '0');
END;
$$;

-- Trigger:
CREATE OR REPLACE FUNCTION trg_set_order_number()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
NEW.order_number := generate_order_number();
RETURN NEW;
END;
$$;

CREATE TRIGGER set_order_number
BEFORE INSERT ON orders
FOR EACH ROW EXECUTE FUNCTION trg_set_order_number();
```

11.2 generate_pickup_code()

Generates a unique 6-character alphanumeric pickup code. Retries on collision.

```
CREATE OR REPLACE FUNCTION generate_pickup_code()
RETURNS TEXT LANGUAGE plpgsql AS $$
DECLARE
chars TEXT := 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
code TEXT;
tries INTEGER := 0;
BEGIN
LOOP
code := '';
FOR i IN 1..6 LOOP
code := code || substr(chars, floor(random()*length(chars)+1)::int, 1);
END LOOP;
EXIT WHEN NOT EXISTS (SELECT 1 FROM orders WHERE pickup_code = code);
tries := tries + 1;
IF tries > 20 THEN RAISE EXCEPTION 'pickup_code: too many collisions'; END IF;
END LOOP;
RETURN code;
END;
$$;
```

11.3 calculate_loyalty_tier(lifetime_points)

Returns the correct tier string based on lifetime points. Called by trigger whenever `loyalty_cards.lifetime_points` is updated.

```
CREATE OR REPLACE FUNCTION calculate_loyalty_tier(p_lifetime_points INTEGER)
RETURNS TEXT LANGUAGE plpgsql STABLE AS $$
DECLARE
    _tier TEXT;
BEGIN
    SELECT tier_name INTO _tier
    FROM loyalty_tiers
    WHERE min_lifetime_points <= p_lifetime_points
    ORDER BY min_lifetime_points DESC
    LIMIT 1;
    RETURN COALESCE(_tier, 'bronze');
END;
$$;
```

11.4 check_discount_code_validity()

Validates a discount code for a specific customer and order total. Returns whether the code is valid, the discount amount in SLSH, and an error message.

```
CREATE OR REPLACE FUNCTION check_discount_code_validity(
    p_code TEXT,
    p_customer_id UUID,
    p_order_total BIGINT
)
RETURNS TABLE(is_valid BOOLEAN, discount_amount_slsh BIGINT, error_message TEXT)
LANGUAGE plpgsql STABLE SECURITY DEFINER AS $$
DECLARE
    dc discount_codes%ROWTYPE;
    uses INTEGER;
BEGIN
    SELECT * INTO dc FROM discount_codes
    WHERE upper(code) = upper(p_code) AND is_active = true
    AND now() BETWEEN valid_from AND valid_until;

    IF NOT FOUND THEN
        RETURN QUERY SELECT false, 0::BIGINT, 'Invalid or expired code';
        RETURN;
    END IF;

    IF dc.max_total_uses IS NOT NULL AND dc.uses_count >= dc.max_total_uses THEN
        RETURN QUERY SELECT false, 0::BIGINT, 'Code has reached its usage limit';
        RETURN;
    END IF;

    SELECT COUNT(*) INTO uses FROM discount_code_uses
    WHERE discount_code_id = dc.id AND customer_id = p_customer_id;

    IF uses >= dc.max_uses_per_customer THEN
        RETURN QUERY SELECT false, 0::BIGINT, 'You have already used this code';
        RETURN;
    END IF;

    IF p_order_total < dc.minimum_order_slsh THEN
        RETURN QUERY SELECT false, 0::BIGINT,
        'Minimum order of ' || dc.minimum_order_slsh || ' SLSH required';
        RETURN;
    END IF;
```

```

END IF;

RETURN QUERY SELECT true,
CASE dc.discount_type
WHEN 'percentage' THEN (p_order_total * dc.value / 100)::BIGINT
ELSE dc.value::BIGINT
END,
NULL::TEXT;
END;
$$;

```

11.5 match_products_semantic()

pgvector semantic search. Returns products ranked by cosine similarity to the query embedding. Used by the /api/ai/search endpoint.

```

CREATE OR REPLACE FUNCTION match_products_semantic(
query_embedding vector(768),
match_limit INTEGER DEFAULT 20,
p_location_id UUID DEFAULT NULL
)
RETURNS TABLE(product_id UUID, similarity FLOAT)
LANGUAGE plpgsql STABLE AS $$
BEGIN
RETURN QUERY
SELECT pe.product_id,
1 - (pe.embedding <=> query_embedding) AS similarity
FROM product_embeddings pe
JOIN products p ON p.id = pe.product_id
WHERE p.is_active = true
AND (p_location_id IS NULL OR EXISTS (
SELECT 1 FROM inventory i
WHERE i.product_id = pe.product_id
AND i.location_id = p_location_id
AND (i.quantity_on_hand - i.quantity_reserved) > 0
))
ORDER BY pe.embedding <=> query_embedding
LIMIT match_limit;
END;
$$;

```

11.6 release_expired_reservations() — pg_cron

Called every 10 minutes. Finds orders older than 30 minutes in 'pending_payment' status and releases reserved stock.

```

CREATE OR REPLACE FUNCTION release_expired_reservations()
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
-- Decrement quantity_reserved for items in expired orders
UPDATE inventory i
SET quantity_reserved = GREATEST(0, quantity_reserved - oi.quantity),
updated_at = now()
FROM order_items oi

```

```

JOIN orders o ON o.id = oi.order_id
WHERE oi.product_id = i.product_id
AND (oi.variant_id = i.variant_id OR (oi.variant_id IS NULL AND i.variant_id IS NULL))
AND o.status = 'pending_payment'
AND o.created_at < now() - interval '30 minutes';

-- Cancel the expired orders
UPDATE orders
SET status = 'cancelled',
    cancellation_reason = 'Payment timeout – auto-cancelled after 30 minutes',
    updated_at = now()
WHERE status = 'pending_payment'
AND created_at < now() - interval '30 minutes';
END;
$$;

-- Schedule with pg_cron (runs every 10 minutes):
SELECT cron.schedule('release-reservations', '*/10 * * * *',
    'SELECT release_expired_reservations()');
```

11.7 expire_parked_transactions() — pg_cron

```

CREATE OR REPLACE FUNCTION expire_parked_transactions()
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
DELETE FROM parked_transactions
WHERE expires_at < now() AND is_recalled = false;
END;
$$;

-- Schedule every 30 minutes:
SELECT cron.schedule('expire-parked-carts', '*/30 * * * *',
    'SELECT expire_parked_transactions()');
```

11.8 generate_pos_transaction_number()

```

CREATE SEQUENCE IF NOT EXISTS pos_txn_seq START 1;
CREATE OR REPLACE FUNCTION generate_pos_transaction_number()
RETURNS TEXT LANGUAGE plpgsql AS $$
BEGIN
RETURN 'POS-' || to_char(now(), 'YYYY') || '-'
|| lpad(nextval('pos_txn_seq')::text, 5, '0');
END;
$$;
```

12. Database Triggers

12.1 Auto-create profile on new auth user

```
CREATE OR REPLACE FUNCTION trg_auto_create_profile()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
INSERT INTO public.profiles (user_id, phone, role, full_name)
VALUES (NEW.id, NEW.phone, 'customer', COALESCE(NEW.raw_user_meta_data->>'full_name',''));
RETURN NEW;
END;
$$;

CREATE TRIGGER auto_create_profile
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION trg_auto_create_profile();
```

12.2 Reserve inventory on order placed

```
CREATE OR REPLACE FUNCTION trg_reserve_inventory()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
-- Raise error if insufficient stock before reserving
IF EXISTS (
SELECT 1 FROM order_items oi
JOIN inventory i ON i.product_id = oi.product_id
AND (i.variant_id = oi.variant_id OR (i.variant_id IS NULL AND oi.variant_id IS NULL))
AND i.location_id = NEW.location_id
WHERE oi.order_id = NEW.id
AND (i.quantity_on_hand - i.quantity_reserved) < oi.quantity
) THEN
RAISE EXCEPTION 'Insufficient stock for one or more items';
END IF;

UPDATE inventory i
SET quantity_reserved = quantity_reserved + oi.quantity,
updated_at = now()
FROM order_items oi
WHERE oi.order_id = NEW.id
AND i.product_id = oi.product_id
AND (i.variant_id = oi.variant_id OR (i.variant_id IS NULL AND oi.variant_id IS NULL))
AND i.location_id = NEW.location_id;
RETURN NEW;
END;
$$;

CREATE TRIGGER reserve_inventory
AFTER INSERT ON orders
FOR EACH ROW EXECUTE FUNCTION trg_reserve_inventory();
```

12.3 Confirm inventory on payment confirmed

```
CREATE OR REPLACE FUNCTION trg_confirm_inventory()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
```

```

BEGIN
IF NEW.status = 'payment_confirmed' AND OLD.status != 'payment_confirmed' THEN
UPDATE inventory i
SET quantity_on_hand = quantity_on_hand - oi.quantity,
quantity_reserved = GREATEST(0, quantity_reserved - oi.quantity),
updated_at = now()
FROM order_items oi
WHERE oi.order_id = NEW.id
AND i.product_id = oi.product_id
AND (i.variant_id = oi.variant_id OR (i.variant_id IS NULL AND oi.variant_id IS NULL))
AND i.location_id = NEW.location_id;
-- Insert stock movement records
INSERT INTO stock_movements
(product_id, variant_id, location_id, movement_type, quantity_change, reference_id, reference_type,
performed_by)
SELECT oi.product_id, oi.variant_id, NEW.location_id, 'sale', -oi.quantity, NEW.id, 'order',
NEW.customer_id
FROM order_items oi WHERE oi.order_id = NEW.id;
END IF;
RETURN NEW;
END;
$$;

CREATE TRIGGER confirm_inventory
AFTER UPDATE OF status ON orders
FOR EACH ROW EXECUTE FUNCTION trg_confirm_inventory();

```

12.4 Release reservation on order cancelled

```

CREATE OR REPLACE FUNCTION trg_release_reservation()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
IF NEW.status = 'cancelled' AND OLD.status NOT IN ('cancelled', 'payment_confirmed', 'completed')
THEN
UPDATE inventory i
SET quantity_reserved = GREATEST(0, quantity_reserved - oi.quantity),
updated_at = now()
FROM order_items oi
WHERE oi.order_id = NEW.id
AND i.product_id = oi.product_id
AND (i.variant_id = oi.variant_id OR (i.variant_id IS NULL AND oi.variant_id IS NULL))
AND i.location_id = NEW.location_id;
END IF;
RETURN NEW;
END;
$$;

CREATE TRIGGER release_reservation
AFTER UPDATE OF status ON orders
FOR EACH ROW EXECUTE FUNCTION trg_release_reservation();

```

12.5 Create loyalty card on new customer

```

CREATE SEQUENCE IF NOT EXISTS loyalty_card_seq START 1;
CREATE OR REPLACE FUNCTION trg_create_loyalty_card()

```

```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
IF NEW.role = 'customer' THEN
INSERT INTO loyalty_cards (customer_id, card_number)
VALUES (NEW.user_id, 'BH-LC-' || lpad(nextval('loyalty_card_seq')::text, 6, '0'));
END IF;
RETURN NEW;
END;
$$;

CREATE TRIGGER create_loyalty_card
AFTER INSERT ON profiles
FOR EACH ROW EXECUTE FUNCTION trg_create_loyalty_card();

```

12.6 Award loyalty points on order/POS completion

```

CREATE OR REPLACE FUNCTION trg_award_loyalty_points()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
_points INTEGER;
_card_id UUID;
BEGIN
IF NEW.status = 'completed' AND OLD.status != 'completed' THEN
_points := floor(NEW.total_slsh / 1000)::INTEGER;
IF _points > 0 AND NEW.customer_id IS NOT NULL THEN
SELECT id INTO _card_id FROM loyalty_cards WHERE customer_id = NEW.customer_id;
IF FOUND THEN
UPDATE loyalty_cards
SET current_points = current_points + _points,
lifetime_points = lifetime_points + _points,
current_tier = calculate_loyalty_tier(lifetime_points + _points),
updated_at = now()
WHERE id = _card_id;

INSERT INTO loyalty_transactions
(loyalty_card_id, transaction_type, points, reference_type, reference_id)
VALUES (_card_id, 'earn', _points, TG_TABLE_NAME, NEW.id);
END IF;
END IF;
END IF;
RETURN NEW;
END;
$$;

-- Attach to both orders and pos_transactions:
CREATE TRIGGER award_loyalty_order
AFTER UPDATE OF status ON orders
FOR EACH ROW EXECUTE FUNCTION trg_award_loyalty_points();

CREATE TRIGGER award_loyalty_pos
AFTER INSERT ON pos_transactions
FOR EACH ROW EXECUTE FUNCTION trg_award_loyalty_points();

```

12.7 Low-stock alert trigger

```

CREATE OR REPLACE FUNCTION trg_check_low_stock()

```

```

RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
_alert_type TEXT;
BEGIN
IF NEW.quantity_on_hand <= NEW.threshold THEN
_alert_type := CASE WHEN NEW.quantity_on_hand = 0 THEN 'out_of_stock' ELSE 'low_stock' END;
INSERT INTO inventory_alerts (product_id, variant_id, location_id, alert_type)
VALUES (NEW.product_id, NEW.variant_id, NEW.location_id, _alert_type)
ON CONFLICT DO NOTHING; -- avoid duplicate active alerts
ELSE
-- Auto-resolve existing alert when stock recovers
UPDATE inventory_alerts
SET is_resolved = true, resolved_at = now()
WHERE product_id = NEW.product_id
AND (variant_id = NEW.variant_id OR (variant_id IS NULL AND NEW.variant_id IS NULL))
AND location_id = NEW.location_id
AND is_resolved = false;
END IF;
RETURN NEW;
END;
$$;

CREATE TRIGGER check_low_stock
AFTER UPDATE OF quantity_on_hand ON inventory
FOR EACH ROW EXECUTE FUNCTION trg_check_low_stock();

```

12.8 Generic audit log trigger

```

CREATE OR REPLACE FUNCTION trg_audit_log()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
INSERT INTO audit_log (table_name, record_id, action, old_data, new_data, performed_by)
VALUES (
TG_TABLE_NAME,
COALESCE(NEW.id, OLD.id),
TG_OP,
CASE WHEN TG_OP = 'INSERT' THEN NULL ELSE to_jsonb(OLD) END,
CASE WHEN TG_OP = 'DELETE' THEN NULL ELSE to_jsonb(NEW) END,
auth.uid()
);
RETURN COALESCE(NEW, OLD);
END;
$$;

-- Apply to each audited table:
CREATE TRIGGER audit_products AFTER INSERT OR UPDATE OR DELETE ON products
FOR EACH ROW EXECUTE FUNCTION trg_audit_log();
CREATE TRIGGER audit_product_variants AFTER INSERT OR UPDATE OR DELETE ON product_variants
FOR EACH ROW EXECUTE FUNCTION trg_audit_log();
CREATE TRIGGER audit_inventory AFTER INSERT OR UPDATE OR DELETE ON inventory
FOR EACH ROW EXECUTE FUNCTION trg_audit_log();
CREATE TRIGGER audit_discount_codes AFTER INSERT OR UPDATE OR DELETE ON discount_codes
FOR EACH ROW EXECUTE FUNCTION trg_audit_log();
CREATE TRIGGER audit_profiles AFTER UPDATE OF role ON profiles
FOR EACH ROW EXECUTE FUNCTION trg_audit_log();
-- (repeat for: categories, locations, loyalty_tiers, mobile_money_settings, banners)

```


12.9 Update POS session totals

```
CREATE OR REPLACE FUNCTION trg_update_session_totals()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
UPDATE pos_sessions
SET total_sales_slsh = (
SELECT COALESCE(SUM(total_slsh),0) FROM pos_transactions
WHERE pos_session_id = NEW.pos_session_id AND status = 'completed'
),
total_voids_slsh = (
SELECT COALESCE(SUM(total_slsh),0) FROM pos_transactions
WHERE pos_session_id = NEW.pos_session_id AND status = 'voided'
),
updated_at = now()
WHERE id = NEW.pos_session_id;
RETURN NEW;
END;
$$;

CREATE TRIGGER update_session_totals
AFTER INSERT OR UPDATE OF status ON pos_transactions
FOR EACH ROW EXECUTE FUNCTION trg_update_session_totals();
```

12.10 Apply stocktake corrections on approval

```
CREATE OR REPLACE FUNCTION trg_apply_stocktake()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
IF NEW.status = 'approved' AND OLD.status != 'approved' THEN
UPDATE inventory i
SET quantity_on_hand = si.counted_quantity, updated_at = now()
FROM stocktake_items si
WHERE si.stocktake_id = NEW.id
AND i.product_id = si.product_id
AND (i.variant_id = si.variant_id OR (i.variant_id IS NULL AND si.variant_id IS NULL))
AND i.location_id = NEW.location_id;
INSERT INTO stock_movements
(product_id, variant_id, location_id, movement_type, quantity_change, reference_id, reference_type,
performed_by)
SELECT si.product_id, si.variant_id, NEW.location_id,
'stocktake_correction', si.discrepancy, NEW.id, 'stocktake', NEW.approved_by
FROM stocktake_items si WHERE si.stocktake_id = NEW.id AND si.discrepancy != 0;
END IF;
RETURN NEW;
END;
$$;

CREATE TRIGGER apply_stocktake
AFTER UPDATE OF status ON stocktakes
FOR EACH ROW EXECUTE FUNCTION trg_apply_stocktake();
```

13. Row-Level Security Policies

RLS is enabled on every table. The service role key bypasses all policies and is used only in server-side Next.js API routes (never exposed to the client). Client-side code uses the anon key and is subject to all policies below.

Table	SELECT	INSERT	UPDATE	DELETE
profiles	Own row (any auth); Staff: same location; Admin: all	Via trigger only	Own row; Admin: any	Admin only
locations	Anyone (public)	Admin only	Admin only	Admin only
location_hours	Anyone (public)	Admin only	Admin only	Admin only
exchange_rates	Anyone (public)	Admin only	—	—
mobile_money_settings	Authenticated	Admin only	Admin only	Admin only
banners	Anyone (public)	Admin only	Admin only	Admin only
categories	Anyone (public)	Admin only	Admin only	Admin only
products	is_active=true: public; All rows: admin/inv_mgr	Admin/inv_mgr	Admin/inv_mgr	Admin only (soft delete)
product_images	Public	Admin/inv_mgr	Admin/inv_mgr	Admin/inv_mgr
product_variants	is_active=true: public; All: admin/inv_mgr	Admin/inv_mgr	Admin/inv_mgr	Admin only
product_embeddings	Authenticated	Service role only	Service role only	Service role only
inventory	Public (qty only); Full: staff/admin	Admin/inv_mgr	Admin/inv_mgr: own location; Admin: any	Admin only
stock_movements	Staff: own location; Admin: all	Service role only	—	—
inventory_alerts	Staff: own location; Admin: all	Via trigger only	Staff: resolve; Admin: all	—
stocktakes	Inv_mgr: own loc; Admin: all	Inv_mgr (own loc); Admin	Inv_mgr (own loc); Admin: approve	—
stocktake_items	Same as stocktakes	Inv_mgr (own loc)	Inv_mgr (own loc)	—
discount_codes	Authenticated (valid only)	Admin only	Admin only	Admin only
discount_code_usages	Service role only	Service role only	—	—
orders	Own: customer; Location: staff; Admin: all	Authenticated (own)	Staff (status); Admin: all	—
order_items	Inherits from orders	Service role only	—	—
payment_transactions	Own: customer; Location: staff; Admin: all	Service role only	Staff: confirm; Admin: all	—
loyalty_tiers	Public	—	Admin only	—

Table	SELECT	INSERT	UPDATE	DELETE
loyalty_cards	Own: customer; Staff: any; Admin: all	Via trigger only	Via trigger only	—
loyalty_transactions	Own: customer; Staff: any; Admin: all	Via trigger only	—	—
pos_sessions	Own: cashier; Admin: all	Cashier (own)	Cashier (own/close); Admin: all	—
pos_transactions	Own session: cashier; Location: staff; Admin: all	Cashier (own session)	Admin/inv_mgr (void)	—
pos_transaction_items	Same as pos_transactions	Cashier (own session)	—	—
pos_payment_splits	Same as pos_transactions	Cashier (own session)	Cashier/staff (confirm)	—
parked_transactions	Own: cashier; Admin: all	Cashier (own session)	Cashier (own)	Cashier (own); Admin
returns	Own: customer; Location: staff; Admin: all	Authenticated (own)	Staff (process); Admin: all	—
return_items	Same as returns	Authenticated (own)	—	—
ai_forecasts	Inv_mgr/admin: own location	Service role only	—	Service role only
audit_log	Admin only	Via trigger only	—	—
sms_logs	Admin only	Service role only	Service role only	—

Sample RLS Policy Implementation

```
-- Enable RLS on all tables (run once per table):
ALTER TABLE products ENABLE ROW LEVEL SECURITY;

-- products: public can read active products
CREATE POLICY products_public_read ON products
FOR SELECT USING (is_active = true);

-- products: admin and inventory_manager can read all (incl. inactive)
CREATE POLICY products_staff_read ON products
FOR SELECT USING (auth.user_role() IN ('admin', 'inventory_manager'));

-- products: admin and inventory_manager can insert
CREATE POLICY products_staff_insert ON products
FOR INSERT WITH CHECK (auth.user_role() IN ('admin', 'inventory_manager'));

-- products: admin and inventory_manager can update
CREATE POLICY products_staff_update ON products
FOR UPDATE USING (auth.user_role() IN ('admin', 'inventory_manager'));

-- orders: customer can only see their own
CREATE POLICY orders_customer_select ON orders
FOR SELECT USING (customer_id = auth.uid());

-- orders: staff can see orders at their location
CREATE POLICY orders_staff_select ON orders
FOR SELECT USING (
```

```
auth.is_staff() AND location_id = auth.user_location_id()
);
-- audit_log: admin read only, no write from any authenticated role
CREATE POLICY audit_log_admin_read ON audit_log
FOR SELECT USING (auth.is_admin());
```

14. Key API Endpoints

All endpoints are Next.js 14 App Router Route Handlers (src/app/api/...). Server-side handlers use the Supabase service role client. Public-facing routes validate with the anon client and rely on RLS.

14.1 Authentication

Method + Path	Auth Required	Description
POST /api/auth/send-otp	None	Sends Twilio OTP to phone number via Supabase Auth
POST /api/auth/verify-otp	None	Verifies OTP, returns Supabase session tokens
POST /api/auth/sign-out	Any auth	Invalidates Supabase session
GET /api/auth/me	Any auth	Returns own profile + role
PATCH /api/auth/profile	Any auth	Update full_name, preferred_language, preferred_currency

14.2 Product Catalog (Customer-facing)

Method + Path	Auth Required	Description
GET /api/products	None	Paginated product list with filters: category, search query, location, sort
GET /api/products/[id]	None	Product detail with variants, images, inventory at requested location
GET /api/categories	None	Full category tree (nested JSON)
POST /api/products/search	None	Semantic search: accepts text query, embeds with Gemini, calls match_products_semantic()
GET /api/products/featured	None	Returns products where is_featured=true for home page

14.3 Orders (Customer Portal)

Method + Path	Auth Required	Description
POST /api/orders	customer	Create order: validates stock, applies discount code, reserves inventory
GET /api/orders	customer	List own orders (paginated)
GET /api/orders/[id]	customer	Get order detail including items and payment status
POST /api/orders/[id]/payment-sent	customer	Customer submits payment reference after mobile money transfer
POST /api/orders/[id]/cancel	customer	Cancel order in pending_payment status; releases reservation

14.4 Returns (Customer + Staff)

Method + Path	Auth Required	Description
POST /api/returns	customer	Submit return request with items and photos

Method + Path	Auth Required	Description
GET /api/returns/[id]	customer	Get return status
PATCH /api/returns/[id]	cashier+	Staff approves/rejects return, sets refund amount and method

14.5 Loyalty

Method + Path	Auth Required	Description
GET /api/loyalty/card	customer	Own loyalty card: points, tier, card number
GET /api/loyalty/transactions	customer	Own points history (paginated)
POST /api/loyalty/redeem-check	customer	Calculate loyalty discount for a given redemption amount before checkout

14.6 AI Services

Method + Path	Auth Required	Description
POST /api/ai/chat	Any auth	AI hardware assistant chatbot — Gemini 1.5 Flash, streams response
POST /api/ai/estimate	Any auth	Project material estimator — Gemini 1.5 Pro, returns itemized list + SLSH total
POST /api/ai/search	None	Semantic product search — embeds query, calls pgvector function
POST /api/ai/generate-description	admin+	Generate bilingual product description from name/attributes via Gemini
POST /api/ai/embed-product	admin+	Trigger (re)embedding of a product's search vectors

14.7 Admin — Products & Catalog

Method + Path	Auth Required	Description
GET /api/admin/products	admin	All products including inactive, with full detail
POST /api/admin/products	admin	Create product with initial variant
PATCH /api/admin/products/[id]	admin	Update product metadata
POST /api/admin/products/[id]/variants	admin	Add variant to product
PATCH /api/admin/variants/[id]	admin	Update variant price, SKU, attributes
POST /api/admin/products/[id]/images	admin	Upload and attach image (Sharp resize → Storage)
DELETE /api/admin/products/[id]	admin	Soft delete: set is_active=false
GET /api/admin/categories	admin	Full category tree including inactive
POST /api/admin/categories	admin	Create category
PATCH /api/admin/categories/[id]	admin	Update category

14.8 Admin — Operations

Method + Path	Auth Required	Description
GET /api/admin/orders	admin	All orders across all locations with filters
PATCH /api/admin/orders/[id]/confirm-payment	admin	Mark payment confirmed, triggers inventory decrement
PATCH /api/admin/orders/[id]/ready	admin	Mark order ready for pickup, sends SMS to customer
PATCH /api/admin/orders/[id]/complete	admin	Mark order completed on pickup
GET /api/admin/staff	admin	List all staff profiles
POST /api/admin/staff	admin	Assign role to existing user by phone
PATCH /api/admin/staff/[id]	admin	Change role or location assignment
GET /api/admin/analytics	admin	Sales totals, top products, revenue by location (date range)
GET /api/admin/analytics/export	admin	CSV export of orders/transactions for date range
POST /api/admin/discounts	admin	Create discount code
PATCH /api/admin/discounts/[id]	admin	Update or deactivate discount code
POST /api/admin/banners	admin	Create promotional banner
PATCH /api/admin/banners/[id]	admin	Update or deactivate banner
POST /api/admin/exchange-rate	admin	Set new USD/SLSH exchange rate

14.9 Inventory Management

Method + Path	Auth Required	Description
GET /api/inventory	inv_mgr+	Stock levels for own location with low-stock filter
PATCH /api/inventory/[id]	inv_mgr+	Update threshold or aisle_shelf
POST /api/inventory/receive	inv_mgr+	Record stock receipt: increments quantity_on_hand, creates stock_movement
POST /api/inventory/transfer	admin	Transfer stock between locations
GET /api/inventory/alerts	inv_mgr+	Active low-stock/out-of-stock alerts for own location
PATCH /api/inventory/alerts/[id]	inv_mgr+	Manually resolve an alert
POST /api/inventory/stocktakes	inv_mgr+	Initiate new stocktake
PATCH /api/inventory/stocktakes/[id]	inv_mgr+	Update stocktake items or submit for approval
PATCH /api/inventory/stocktakes/[id]/approve	admin	Approve stocktake — triggers inventory correction
GET /api/inventory/forecasts	inv_mgr+	AI reorder forecasts for own location

14.10 POS & Staff

Method + Path	Auth Required	Description
POST /api/pos/sessions	cashier+	Open a new POS session (shift start)

Method + Path	Auth Required	Description
PATCH /api/pos/sessions/[id]/close	cashier+	Close session with ending cash count
GET /api/pos/sessions/active	cashier+	Get own open session
POST /api/pos/transactions	cashier+	Complete a POS sale: decrements inventory, awards loyalty
POST /api/pos/transactions/[id]/void	cashier+	Void a transaction with reason
POST /api/pos/park	cashier+	Park current cart
GET /api/pos/parked	cashier+	List own parked carts
POST /api/pos/parked/[id]/recall	cashier+	Recall and delete parked cart
POST /api/pos/sync	cashier+	Offline sync: batch-submit transactions collected offline
POST /api/pos/pickup/verify	cashier+	Verify pickup code and mark order completed
POST /api/pos/payment/confirm	cashier+	Confirm mobile money payment for online order
GET /api/pos/customer-lookup	cashier+	Lookup customer by phone: loyalty card + open orders
GET /api/pos/products/barcode/[sku]	cashier+	Barcode scan lookup: return variant + price + stock

15. Storage Buckets & Environment Variables

15.1 Supabase Storage Buckets

Bucket Name	Public?	Contents	Max File Size
product-images	Yes	Full-resolution product photos (Sharp-resized to max 1200px)	5 MB
product-thumbnails	Yes	Thumbnail versions (300x300, Sharp WebP)	500 KB
category-icons	Yes	Category SVG/PNG icons	200 KB
banners	Yes	Promotional banner images	2 MB
return-photos	No (auth)	Customer-uploaded photos for return requests	10 MB
receipts	No (auth)	Generated PDF receipts (jsPDF)	1 MB

15.2 Required Environment Variables

```
# .env.local – never commit to source control
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ... # server-side only, never NEXT_PUBLIC_
# Google Gemini
GEMINI_API_KEY=AIZA...
# Upstash Redis (rate limiting)
UPSTASH_REDIS_REST_URL=https://...upstash.io
UPSTASH_REDIS_REST_TOKEN=AX...
# Twilio (SMS – optional if using Supabase built-in Twilio)
TWILIO_ACCOUNT_SID=AC...
TWILIO_AUTH_TOKEN=...
TWILIO_PHONE_NUMBER=+1...
# App
NEXT_PUBLIC_APP_URL=https://borama-hardware.vercel.app
NEXT_PUBLIC_DEFAULT_LOCATION_ID=uuid-of-main-branch
```

15.3 Required PostgreSQL Extensions

```
-- Run once in Supabase SQL editor:
CREATE EXTENSION IF NOT EXISTS "uuid-ossf"; -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS vector; -- pgvector for embeddings
CREATE EXTENSION IF NOT EXISTS pg_cron; -- scheduled jobs
CREATE EXTENSION IF NOT EXISTS pg_trgm; -- trigram index for ILIKE search
-- Trigram index for fast keyword search on products:
CREATE INDEX idx_products_name_trgm ON products
USING GIN (name_en gin_trgm_ops, name_so gin_trgm_ops);
```

```
-- IVFFlat index for fast vector similarity search:  
CREATE INDEX idx_embeddings_ivfflat ON product_embeddings  
USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```